

Can you tell me how CompositeBackend for deepagents works and give me a simple example

Worked for 12s >

Yes. The simplest mental model is:

`CompositeBackend` is a router for the agent's virtual filesystem.

Instead of saying, "all file operations go to this one backend," you can say:

"Anything under `/skills/` goes to this backend; everything else goes somewhere else."

Deep Agents routes based on the **longest matching path prefix**, and unmatched paths go to the `default` backend.  Docs by LangCh...

Simple example

Suppose you physically have:

```
/my/home/dir/skills/
├─ math/
│   └─ SKILL.md
└─ quotes/
    └─ SKILL.md
```

You could do this:

```
from deepagents import create_deep_agent
from deepagents.backends import (
    CompositeBackend,
    FilesystemBackend,
    StateBackend,
)

backend = CompositeBackend(
    default=StateBackend(),
    routes={
```

```

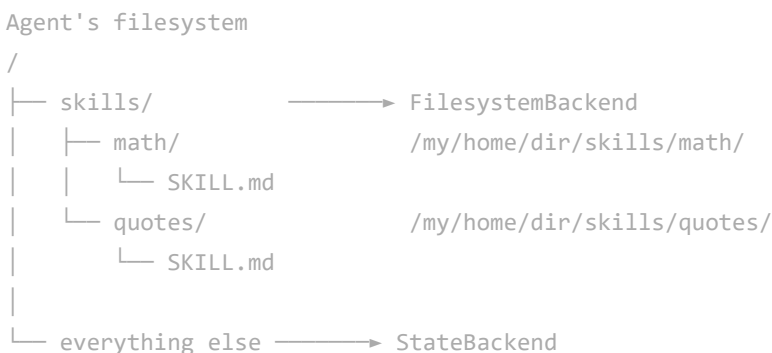
        "/skills/": FilesystemBackend(
            root_dir="/my/home/dir/skills",
            virtual_mode=True,
        ),
    },
)

agent = create_deep_agent(
    model=model,
    backend=backend,
    skills=["/skills/"],
)

```

Now there are **two different filesystem namespaces** behind what looks like one filesystem to the agent. This is exactly what `CompositeBackend` is designed to do. [Docs by LangCh...](#)

Conceptually:



So when Deep Agents asks for:

```
/skills/math/SKILL.md
```

`CompositeBackend` sees the `/skills/` prefix and sends the operation to that `FilesystemBackend`. The route prefix is handled by the composite layer, so the child backend effectively works relative to its own root. Composite preserves the virtual prefixes when returning listings/search results to the agent. [Docs by LangCh...](#)

This separation is useful because Deep Agents also uses its backend for internal files such as offloaded large tool results and conversation history. LangChain specifically recommends wrapping a `FilesystemBackend` in a `CompositeBackend` in many cases so those internal files don't automatically get written into your project/skills directory.

[Docs by LangCh...](#)

For example:

```
/skills/math/SKILL.md
```

goes to:

```
FilesystemBackend
```

```
↓
```

```
/my/home/dir/skills/math/SKILL.md
```

while something like:

```
/large_tool_results/result.txt
```

goes to:

```
StateBackend
```

and therefore doesn't get dumped into:

```
/my/home/dir/skills/
```

Why this is especially nice for skills

It also makes the `skills=` argument much cleaner.

Instead of trying to make:

Python

Run

mean both "the root of my backend" and "the location of my skills," you give the skills an explicit virtual location:

Python

Run

and route that location wherever you want:

```

routes={
    "/skills/": FilesystemBackend(
        root_dir="/my/home/dir/skills",
        virtual_mode=True,
    )
}

```

Deep Agents' Skills middleware scans the configured skill paths at startup, initially loading the name and description metadata from each `SKILL.md`, then reading the full skill only when activated.  Docs by LangCh...

So I would think about the three pieces independently:

```

FilesystemBackend(
    root_dir="/my/home/dir/skills"
)

```

means:

“Where are the real files?”

```

routes={
    "/skills/": filesystem_backend
}

```

means:

“Where should those files appear in the agent's virtual filesystem?”

and:

Python

Run

means:

“Which virtual directories should Deep Agents scan for skills?”

That separation is the main benefit of `CompositeBackend`.



Would this fix the Windows bug that was reported with `deep agents skills=` parameter?

Worked for 20s >

Yes — for the Windows bug you were describing, I think `CompositeBackend` is a very good workaround, and likely the cleanest cross-platform approach.

The reason is important: the reported Windows bug is not really that `skills=` itself is fundamentally broken. It's that Deep Agents can end up exposing/resolving a **Windows physical path** like:

```
D:\my\project\skills\math\SKILL.md
```

and then its filesystem middleware rejects it because Deep Agents expects agent-facing filesystem paths to be virtual paths beginning with `/`. That exact failure appears in Windows issue #1502:

```
ValueError: Windows absolute paths are not supported:
D:\...
Please use virtual paths starting with /
```

The reporter specifically observed that `SkillMiddleware` turned the relative skill path into a Windows absolute path before the agent tried to read it.  [GitHub](#)

With `CompositeBackend`, you can prevent the agent/skills middleware from ever needing to deal with `D:\...` or `/home/...`.

For example:

Python

Run

```
from deepagents import create_deep_agent
from deepagents.backends import (
    CompositeBackend,
    FilesystemBackend,
    StateBackend,
```

```

)

skills_dir = "/my/home/dir/skills"

backend = CompositeBackend(
    default=StateBackend(),
    routes={
        "/skills/": FilesystemBackend(
            root_dir=skills_dir,
            virtual_mode=True,
        ),
    },
)

agent = create_deep_agent(
    model=model,
    backend=backend,
    skills=["/skills/"],
)

```

On Windows, `skills_dir` might instead be:

Python

Run

```
skills_dir = r"C:\Users\selvyn\myapp\skills"
```

but nothing else changes:

Python

Run

```

backend = CompositeBackend(
    default=StateBackend(),
    routes={
        "/skills/": FilesystemBackend(
            root_dir=r"C:\Users\selvyn\myapp\skills",
            virtual_mode=True,
        ),
    },
)

agent = create_deep_agent(
    model=model,
    backend=backend,
    skills=["/skills/"],
)

```

The critical difference is that Deep Agents sees this:

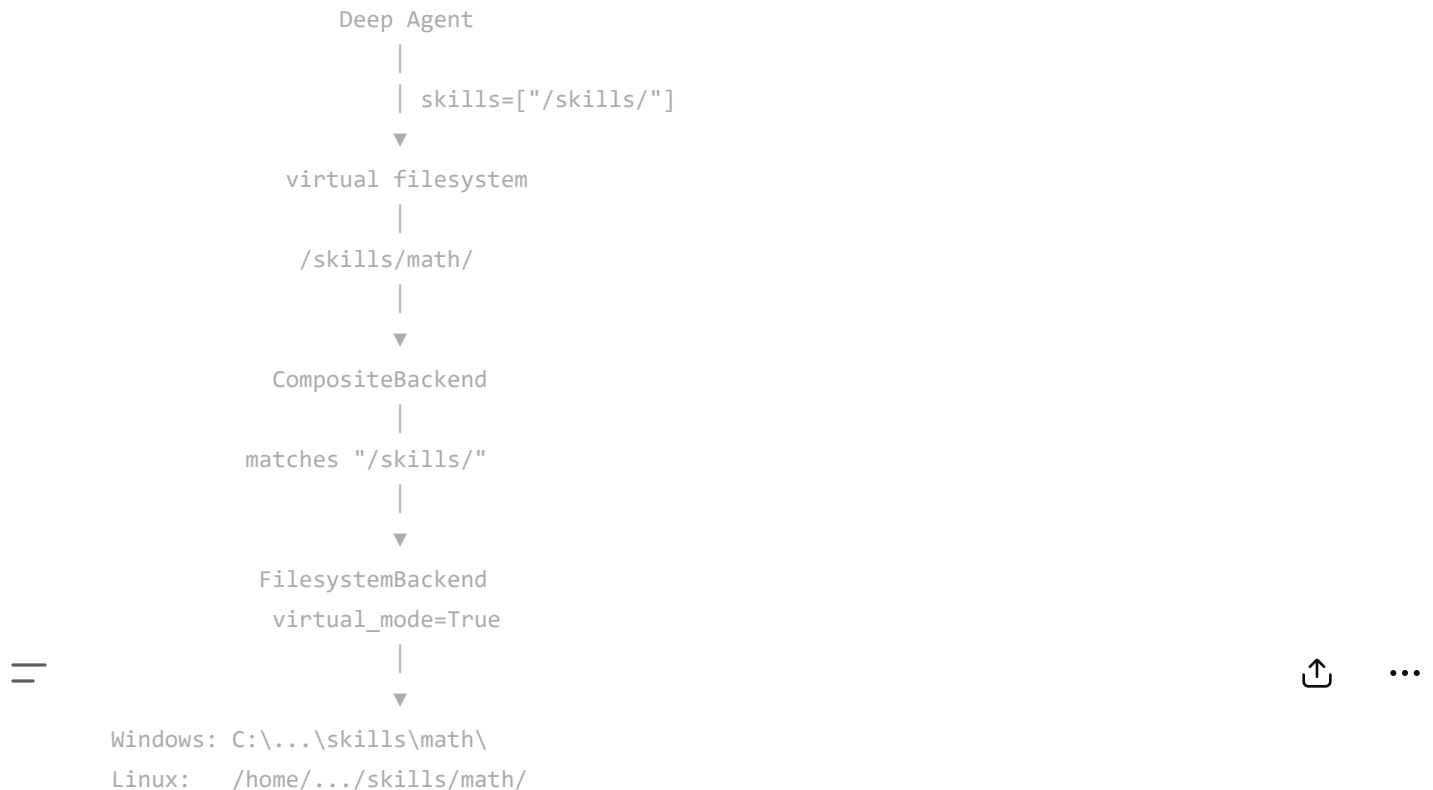
```
/skills/math/SKILL.md
```

rather than:

```
C:\Users\selvyn\myapp\skills\math\SKILL.md
```

CompositeBackend sees `/skills/` and routes it to the `FilesystemBackend`.
`virtual_mode=True` then interprets paths relative to its physical `root_dir`. This is explicitly one of the intended uses of virtual mode with `CompositeBackend`; the current source/docs describe it as allowing `CompositeBackend` to strip routing prefixes and pass normalized virtual paths to the underlying backend. [GitHub](#) +1

So your architecture effectively becomes:



That is much cleaner than making the agent reason about the host OS path.

Compared with your `skills=["/"]` workaround

Previously you were considering something like:

Python

Run

```

backend = FilesystemBackend(
    root_dir=skills_dir,
    virtual_mode=True,
)

agent = create_deep_agent(
    ...
    backend=backend,
    skills=["/"],
)

```

That makes sense why it works:

```

physical root
C:\whatever\skills
    ↓
virtual root
/
    ↓
skills=["/"]

```

So:

```
/math/SKILL.md
```

maps to:

```
C:\whatever\skills\math\SKILL.md
```

But it also means **the entire agent filesystem root is your skills folder**, which is the concern we discussed.

CompositeBackend lets you retain the useful part:

```
physical OS paths hidden behind virtual paths
```

without making your skills directory equal to the agent's entire filesystem.

I'd therefore prefer:

Python

Run

```

CompositeBackend(
    default=StateBackend(),

```



```

routes={
    "/skills/": FilesystemBackend(
        root_dir=skills_dir,
        virtual_mode=True,
    )
}

skills=["/skills/"]

```

over:

Python

Run

```

FilesystemBackend(
    root_dir=skills_dir,
    virtual_mode=True,
)

skills=["/"]

```

One caveat

There is a **different** CompositeBackend issue worth knowing about. If a skill contains executable scripts and tells the agent:

```
python /skills/foo/script.py
```

the filesystem operations can resolve `/skills/...`, but shell `execute()` does not necessarily route virtual paths through `CompositeBackend`. That has been reported separately as issue #3050.  [GitHub](#)

So for ordinary skills consisting of:

```

skills/
├─ math/
│   └─ SKILL.md
└─ quote-count/
    └─ SKILL.md

```

yes, I'd use `CompositeBackend` as the cross-platform solution.

If your skills eventually contain:

```
skills/  
└─ something/  
    └─ SKILL.md  
        └─ scripts/  
            └─ run.py
```

and the LLM needs to execute those scripts, we'd need to think about that second issue separately.

So I would actually change my recommendation from our earlier discussion: rather than relying on `skills=["/"]`, I'd use the explicit virtual `/skills/` route through `CompositeBackend`. It gives you the same Windows-path isolation while keeping the filesystem architecture much cleaner.  GitHub +1



ChatGPT can make mistakes. Check important info.